

Perceptron Loss Functions

1 Intro

- **Loss function** = "How bad was my prediction?"
 - It gives a number (error) that tells us how far the model's prediction is from the true label.
 - The goal of training: **minimize the loss**.
-

2 Recap

Earlier, we saw:

- Perceptron makes predictions by weighted sum + step function.
- If wrong → update weights ("perceptron trick").

But this had a **problem**...

3 Problem with Perceptron Trick

- Step function (hard cutoff: 0 or 1) makes it impossible to measure **how wrong** the prediction was.
- We only know: *right or wrong*.
- But we want a **smooth measure of error**, so we can use calculus (gradients) to improve weights effectively.

That's why **loss functions** are introduced.

4 Loss Function

A **loss function** converts prediction vs reality into a number we can minimize.

General training loop:

1. Make prediction.
 2. Compute loss.
 3. Use gradient descent to reduce loss.
-

5 Perceptron Loss Function

Original perceptron loss:

- If correctly classified → loss = 0.
- If misclassified → loss = some positive penalty.

But it's **not smooth**, which makes optimization harder.

6 SkLearn Documentation

- Libraries like **scikit-learn** implement perceptrons with more stable loss functions.
 - Example: Hinge loss (used in SVMs).
 - This ensures better convergence compared to raw perceptron loss.
-

7 Explanation of Loss Functions

1. Hinge Loss

Formula:

$$L = \max(0, 1 - y \cdot f(x))$$

- y = true label (+1 or -1).
 - $f(x)$ = predicted score.
 - Encourages not just correct classification, but also a **margin** (confidence gap).
 - Used in **SVMs**.
-

2. Binary Cross-Entropy (a.k.a. Log Loss)

Used with **sigmoid activation** (outputs probability).

Formula:

$$L = -[y \cdot \log(p) + (1 - y) \cdot \log(1 - p)]$$

- y = true label (0 or 1).
 - p = predicted probability (between 0 and 1).
 - Very common in modern binary classification.
-

3. Sigmoid Function

Formula:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Converts any real number into a probability (0–1).
 - Smooth and differentiable (better than step function).
 - Pairs naturally with binary cross-entropy.
-

8 Gradient Descent

- Gradient descent uses the **slope of the loss function** to adjust weights.
 - Smooth losses (like hinge, cross-entropy) make optimization possible.
 - Without them, perceptron would just guess & adjust blindly.
-

9 Code Demo

Typical modern binary classification:

```
import numpy as np

# sigmoid
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# binary cross entropy loss
def loss(y, y_pred):
    return - (y*np.log(y_pred) + (1-y)*np.log(1-y_pred))

# example
x = np.array([0.5, 1.2]) # input
w = np.array([0.4, -0.7]) # weights
b = 0.1

z = np.dot(x, w) + b
y_pred = sigmoid(z)

y_true = 1
print("Predicted:", y_pred)
print("Loss:", loss(y_true, y_pred))
```

10 More Loss Functions

Beyond perceptron, hinge, and cross-entropy, there are many:

- **Mean Squared Error (MSE):** regression tasks.
- **Categorical Cross-Entropy:** multi-class classification.
- **Huber Loss:** balances between MSE and MAE.

1 1 Outro

- Loss functions are **the heart of training** — they guide the model's improvement.
 - The perceptron started simple (0/1 error), but modern ML uses smoother, more informative functions (hinge, cross-entropy).
 - Without them, deep learning wouldn't be possible.
-

Quick Recap

- **Problem:** Perceptron trick only says “wrong/correct.”
 - **Solution:** Use loss functions that measure *how wrong*.
 - **Hinge Loss:** margin-based, used in SVMs.
 - **Binary Cross-Entropy:** probability-based, standard in classification.
 - **Sigmoid:** smooth activation, enables probability predictions.
-